



ENSIASD

Rapport tp 4 mini-projet

Réalisées par :

DIDI Aya

Surveillée par :

Pr. A.Aferhane

2025/2026

Table des matières

Partie 1 : Préparation du dataset	4
1. Chargement & nettoyage des captions.....	4
a) Nettoyage et normalisation des données	4
b) Ajout de balises de contrôle	4
c) Structuration des données	4
2. Construction du vocabulaire	5
a) Dictionnaire de mots (Indexing)	5
b) Gestion des tokens spéciaux	5
c) Sauvegarde et cohérence des données	5
3. Split Train / Val / Test	6
4. Transformations des images	6
a) Augmentation de données (Training Set)	6
b) standardisation et normalisation.....	7
5. Dataset & DataLoader	7
a) Classe personnalisée <i>FlickrDataset</i>	8
b) Fonction de regroupement (<i>collate_fn</i>) et padding	8
c) Configuration des DataLoaders	8
Partie 2 : Architecture du Modèle.....	9
1. Encodeur CNN	9
2. Décodeur RNN	10
3. Décodeur Transformer	11
Partie 3 : Entraînement	12
1. Utilitaires d'entraînement	12
2. Boucle d'entraînement principale.....	12
Partie 4 — Évaluation & Résultats	15
1. Génération de légendes	15
Partie 5 : Visualisation des Résultats.....	17
Partie 6 — Questions du TP.....	18
a) Architecture du modèle	18
b) Paramètres d'optimisation.....	18



1. LSTM vs GRU.....	19
2. Tailles d'embedding : 128 / 256 / 512 / 1024	20
3. ResNet18 vs ResNet50	21
4. Impact du Dropout	22
5. Nombre de couches RNN	23
6. Optimiseur : Adam vs SGD	25
a) A. Efficacité d'Adam	25
b) B. Échec relatif de SGD	26
7. CNN + Transformer Decoder	26
Partie 7 : test	27
Conclusion	29

Partie 1 : Préparation du dataset

1. Chargement & nettoyage des captions

```
def load_captions(path: Path) -> dict:
    """
    Lit captions.txt (format : image#N\tcaption)
    """
    captions = {}
    table = str.maketrans('', '', string.punctuation)
    with open(path, 'r') as f:
        # Si votre fichier est un CSV (virgules), on remplace dynamiquement pour votre logique
        content = f.read().replace(',', '\t')

    for line in content.splitlines():
        line = line.strip()
        # On saute l'en-tête si elle existe (image\tcaption)
        if not line or '\t' not in line or line.startswith('image'):
            continue

        img, caption = line.split('\t', 1)
        img = img.split('#')[0].strip()
        caption = caption.lower().translate(table).strip()
        caption = "startseq " + caption + " endseq"
        captions.setdefault(img, []).append(caption)

    return captions
```

Chargement des captions ...
8,091 images chargées
Exemple : ('1000268201_693b08cb0e.jpg', ['startseq a child in a pink dress is climbing up a set of stairs in an entry way endseq'])

a) Nettoyage et normalisation des données

Le texte des légendes est prétraité afin de simplifier son vocabulaire pour le modèle d'intelligence artificielle. Cela inclut la conversion en minuscules pour uniformiser le texte, ainsi que la suppression de la ponctuation afin d'éliminer les caractères non pertinents pour l'apprentissage.

b) Ajout de balises de contrôle

Chaque légende est encadrée par deux tokens spéciaux :

- **startseq** : marque le début de la séquence.
- **endseq** : indique la fin de la description.

Ces balises aident le modèle à apprendre quand commencer et arrêter la génération de texte.

c) Structuration des données

Les données sont organisées sous forme de dictionnaire Python, où chaque clé représente une image unique et chaque valeur contient la liste des légendes associées. Cette structure facilite l'accès rapide aux données lors de l'entraînement du modèle.

2. Construction du vocabulaire

```

vocab_set = set()
for img_caps in all_captions.values():
    for c in img_caps:
        vocab_set.update(word_tokenize(c))

word2idx = {w: i + 1 for i, w in enumerate(sorted(vocab_set))}
word2idx["<pad>"] = 0
idx2word = {i: w for w, i in word2idx.items()}
vocab_size = len(word2idx)
print(f"📄 Taille du vocabulaire : {vocab_size:,} mots")

# Indices spéciaux
PAD_IDX = word2idx["<pad>"]
SOS_IDX = word2idx.get("startseq", 1)
EOS_IDX = word2idx.get("endseq", 2)

# — Sauvegarde du vocabulaire —
vocab_path = OUT_ROOT / 'vocabulary.pkl'
with open(vocab_path, 'wb') as f:
    pickle.dump({'word2idx': word2idx, 'idx2word': idx2word}, f)
print(f"📄 Vocabulaire sauvegardé → {vocab_path}")

```

.. 📄 Taille du vocabulaire : 8,830 mots
 📄 Vocabulaire sauvegardé → /content/drive/MyDrive/Flickr8k_Output/vocabulary.pkl

a) Dictionnaire de mots (Indexing)

Le code construit un vocabulaire à partir de toutes les légendes en attribuant un identifiant numérique unique à chaque mot. Deux structures sont créées :

- **word2idx** : associe chaque mot à un index (ex : "chat" → 42).
- **idx2word** : dictionnaire inverse permettant de transformer les sorties du modèle en texte lisible.

b) Gestion des tokens spéciaux

Des tokens spécifiques sont ajoutés pour faciliter l'apprentissage du modèle :

- **(padding)** : index 0, utilisé pour uniformiser la longueur des séquences.
- **startseq (SOS)** : marque le début d'une légende.
- **endseq (EOS)** : indique la fin de la génération.

c) Sauvegarde et cohérence des données

La taille du vocabulaire est calculée ($\approx 8\,830$ mots), puis les dictionnaires sont enregistrés dans un fichier **vocabulary.pkl** via *pickle*. Cela permet de conserver le même encodage lors de l'inférence et d'assurer la cohérence des prédictions sur de nouvelles images.

3. Split Train / Val / Test

```
all_images = list(all_captions.keys())
random.shuffle(all_images)
n = len(all_images)
train_imgs = set(all_images[:int(0.80 * n)])
val_imgs = set(all_images[int(0.80 * n):int(0.90 * n)])
test_imgs = set(all_images[int(0.90 * n):])
print(f"📊 Train={len(train_imgs)} | Val={len(val_imgs)} | Test={len(test_imgs)}")
```

📊 Train=6472 | Val=809 | Test=810

- **Entraînement (Train - 80%)** : 6 472 images. C'est l'ensemble utilisé par le modèle pour apprendre à associer les caractéristiques visuelles aux mots.
- **Validation (Val - 10%)** : 809 images. Cet ensemble sert à ajuster les hyperparamètres pendant l'entraînement et à surveiller la progression sans que le modèle n'apprenne directement dessus.
- **Test (Test - 10%)** : 810 images. Cet ensemble est maintenu totalement invisible pour le modèle jusqu'à la fin du projet. Il sert à fournir une évaluation finale et impartiale de la capacité du modèle à générer des légendes pour des images qu'il n'a jamais rencontrées.

4. Transformations des images

```
transform_train = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(brightness=0.1, contrast=0.1),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406),
                          (0.229, 0.224, 0.225)),
])

transform_test = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406),
                          (0.229, 0.224, 0.225)),
])
print("✅ Transformations définies")
```

.. ✅ Transformations définies

a) Augmentation de données (Training Set)

Pour améliorer la robustesse du modèle et réduire le surapprentissage, des techniques d'augmentation sont appliquées uniquement sur le jeu d'entraînement. Cela inclut :



- **RandomHorizontalFlip** : inversion horizontale aléatoire des images afin que le modèle apprenne l'invariance gauche/droite.
- **ColorJitter** : variation légère de la luminosité et du contraste pour simuler différentes conditions d'éclairage.

b) tandardisation et normalisation

Certaines transformations sont appliquées à la fois sur l'entraînement et le test pour garantir la cohérence des données :

- **Resize (224×224)** : redimensionnement des images à une taille fixe compatible avec les architectures de vision par ordinateur.
- **ToTensor** : conversion des images en tenseurs PyTorch avec normalisation des pixels entre 0 et 1.
- **Normalize** : standardisation des canaux RGB avec les statistiques d'ImageNet (moyenne et écart-type), ce qui favorise une convergence plus rapide et stable du modèle.

5. Dataset & DataLoader

```
class FlickrDataset(Dataset):
    """
    Un sample = (image_tensor, caption_tensor).
    Une caption par ligne (5 captions x N images = 5N lignes).
    """
    def __init__(self, image_names, captions_dict,
                 images_dir, word2idx, transform=None):
        self.images_dir = Path(images_dir)
        self.word2idx = word2idx
        self.transform = transform
        self.data = []
        for img in image_names:
            for cap in captions_dict.get(img, []):
                tokens = word_tokenize(cap)
                ids = [word2idx.get(w, PAD_IDX) for w in tokens]
                self.data.append((img, ids))

    def __len__(self): return len(self.data)

    def __getitem__(self, idx):
        img_name, ids = self.data[idx]
        image = Image.open(self.images_dir / img_name).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, torch.tensor(ids, dtype=torch.long)

# Méthode utilisée par le TP pour l'échantillonnage par longueur
def get_train_indices(self):
    lengths = [len(ids) for _, ids in self.data]
    target = random.choice(lengths)
    return [i for i, (_, ids) in enumerate(self.data)
            if len(ids) == target]
```

```
def collate_fn(batch):
    """Padding des captions à la même longueur dans un batch."""
    images, captions = zip(*batch)
    images = torch.stack(images, 0)
    lengths = [len(c) for c in captions]
    padded = torch.full((len(captions), max(lengths)),
                        PAD_IDX, dtype=torch.long)
    for i, cap in enumerate(captions):
        padded[i, :len(cap)] = cap
    return images, padded

def get_loader(image_set, batch_size=64, train=True):
    ds = FlickrDataset(image_set, all_captions, IMAGES_DIR,
                       word2idx,
                       transform=transform_train if train else transform_test)
    return DataLoader(ds, batch_size=batch_size,
                     shuffle=train, collate_fn=collate_fn,
                     num_workers=2, pin_memory=True)

train_loader = get_loader(train_imgs, batch_size=64, train=True)
val_loader = get_loader(val_imgs, batch_size=64, train=False)
test_loader = get_loader(test_imgs, batch_size=1, train=False)
print(f"✅ DataLoaders créés")
print(f"  train={len(train_loader.dataset):,} | "
      f"val={len(val_loader.dataset):,} | "
      f"test={len(test_loader.dataset):,} samples")

✅ DataLoaders créés
train=32,360 | val=4,045 | test=4,050 samples
```

a) Classe personnalisée *FlickrDataset*

Cette classe hérite de `torch.utils.data.Dataset` et sert à préparer les données pour l'entraînement du modèle.

- **Initialisation** : chaque image est associée à ses 5 légendes, lesquelles sont converties en séquences d'indices grâce au dictionnaire `word2idx`.
- **getitem** : charge une image selon son index, applique les transformations (resize, normalisation, etc.), puis retourne le couple (*image, séquence de tokens*).

b) Fonction de regroupement (*collate_fn*) et padding

Les légendes ayant des longueurs variables, une étape de normalisation est nécessaire pour les traiter par lots.

La fonction `collate_fn` applique le **padding** afin d'égaliser la longueur des séquences dans un même batch en complétant avec le token `PAD_IDX` (0). Cela permet de former des tenseurs compatibles avec le GPU.

c) Configuration des DataLoaders

Les `DataLoader` assurent la gestion efficace des données :



- **Entraînement** : mélange des données (shuffle=True) et chargement parallèle (num_workers=2) pour accélérer le processus.
- **Validation et test** : chargement séquentiel afin d'obtenir une évaluation stable et reproductible des performances du modèle.

Partie 2 : Architecture du Modèle

1. Encodeur CNN

```
class EncoderCNN(nn.Module):
    """
    ResNet50 pré-entraîné dont la dernière couche FC est remplacée
    par une projection linéaire vers embed_size.
    Les paramètres du backbone sont gelés.
    """
    def __init__(self, embed_size: int, backbone: str = 'resnet50'):
        super().__init__()
        resnet = getattr(models, backbone)(pretrained=True)
        # Désactiver l'apprentissage du backbone
        for param in resnet.parameters():
            param.requires_grad_(False)
        # Retirer la dernière couche de classification
        modules = list(resnet.children())[:-1]
        self.resnet = nn.Sequential(*modules)
        self.embed = nn.Linear(resnet.fc.in_features, embed_size)
        self.bn = nn.BatchNorm1d(embed_size)

    def forward(self, images):
        features = self.resnet(images)          # (B, 2048, 1, 1)
        features = features.view(features.size(0), -1) # (B, 2048)
        features = self.embed(features)         # (B, embed_size)
        return self.bn(features)

print("✅ EncoderCNN défini")
```

2. Décodeur RNN

```
class DecoderRNN(nn.Module):
    """
    Embedding → LSTM (ou GRU) → Linear → scores vocabulaire.

    La feature image est concaténée en tête de la séquence
    d'embeddings (comme dans le TP original).
    """
    def __init__(self, embed_size: int, hidden_size: int,
                  vocab_size: int, num_layers: int = 1,
                  dropout: float = 0.0, rnn_type: str = 'lstm'):
        super().__init__()
        self.hidden_dim = hidden_size
        self.rnn_type = rnn_type

        # Couche d'embedding
        self.embed = nn.Embedding(vocab_size, embed_size,
                                   padding_idx=PAD_IDX)
        self.drop = nn.Dropout(dropout)

        # LSTM ou GRU
        rnn_cls = nn.LSTM if rnn_type == 'lstm' else nn.GRU
        self.lstm = rnn_cls(embed_size, hidden_size,
                             num_layers=num_layers,
                             dropout=dropout if num_layers > 1 else 0,
                             batch_first=True)

        # Couche linéaire finale
        self.linear = nn.Linear(hidden_size, vocab_size)

        # État caché initial (mis à jour durant l'inférence)
        self.hidden = (torch.zeros(1, 1, hidden_size),
                       torch.zeros(1, 1, hidden_size))

    def forward(self, features, captions):
        """
        features : (B, embed_size)
        captions : (B, T) → on retire le dernier token (<end>)
        Retourne : logits (B, T, vocab_size)
        """
        # Embedding des captions (sans le dernier token)
        cap_embedding = self.drop(self.embed(captions[:, :-1])) # (B,T-1,E)
        # Concaténer la feature image en tête
        embeddings = torch.cat(
            (features.unsqueeze(1), cap_embedding), dim=1) # (B,T,E)
        lstm_out, self.hidden = self.lstm(embeddings) # (B,T,H)
        outputs = self.linear(lstm_out) # (B,T,V)
        return outputs

    @torch.no_grad()
    def sample(self, inputs, states=None, max_len: int = 20) -> list:
        """
        Génère une caption mot par mot (décodage greedy).
        inputs : (1, 1, embed_size) – feature image
        """
        self.eval()
        res = []
        for _ in range(max_len):
            lstm_out, states = self.lstm(inputs, states) # (1,1,H)
            outputs = self.linear(lstm_out.squeeze(1)) # (1,V)
            _, pred = outputs.max(dim=1)
            idx = pred.item()
            res.append(idx)
            if idx == EOS_IDX:
                break
            inputs = self.drop(self.embed(pred)).unsqueeze(1) # (1,1,E)
        return res
```

3. Décodeur Transformer

```
class PositionalEncoding(nn.Module):
    def __init__(self, d_model: int, max_len: int = 512, dropout: float = 0.1):
        super().__init__()
        self.dropout = nn.Dropout(dropout)
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len).unsqueeze(1).float()
        div = torch.exp(torch.arange(0, d_model, 2).float()
                        * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))

    def forward(self, x):
        return self.dropout(x + self.pe[:, :x.size(1)])

class TransformerCaptionDecoder(nn.Module):
    """
    nn.TransformerDecoder conditionné sur la feature CNN
    (traitée comme un token mémoire unique).
    """
    def __init__(self, embed_size: int, vocab_size: int,
                 num_heads: int = 8, num_layers: int = 3,
                 ff_dim: int = 2048, dropout: float = 0.1):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_size, padding_idx=PAD_IDX)
        self.pos_enc = PositionalEncoding(embed_size, dropout=dropout)
        layer = nn.TransformerDecoderLayer(
            embed_size, num_heads, ff_dim, dropout, batch_first=True)
        self.transformer = nn.TransformerDecoder(layer, num_layers)
        self.fc = nn.Linear(embed_size, vocab_size)

    def _causal_mask(self, sz, device):
        return torch.triu(torch.ones(sz, sz, device=device), diagonal=1).bool()

    def forward(self, features, captions):
        memory = features.unsqueeze(1)  # (B,1,E)
        tgt = self.pos_enc(self.embed(captions[:, :-1]))  # (B,T-1,E)
        mask = self._causal_mask(tgt.size(1), tgt.device)
        pad_mask = (captions[:, :-1] == PAD_IDX)
        out = self.transformer(tgt, memory,
                              tgt_mask=mask,
                              tgt_key_padding_mask=pad_mask)
        return self.fc(out)  # (B,T-1,V)

    @torch.no_grad()
    def sample(self, feature, max_len: int = 20) -> list:
        self.eval()
        tokens = torch.tensor([[SOS_IDX]], device=feature.device)
        memory = feature.unsqueeze(1)
        res = []
        for _ in range(max_len):
            tgt = self.pos_enc(self.embed(tokens))
            msk = self._causal_mask(tgt.size(1), tgt.device)
            out = self.transformer(tgt, memory, tgt_mask=msk)
            nxt = self.fc(out[:, -1]).argmax(dim=-1, keepdim=True)
            idx = nxt.item()
            if idx == EOS_IDX:
                break
            res.append(idx)
            tokens = torch.cat([tokens, nxt], dim=1)
        return res
```

Partie 3 : Entraînement

1. Utilitaires d'entraînement

```
class Logger:
    """Logger qui écrit dans un fichier Drive ET affiche dans la console."""
    def __init__(self, path: Path):
        self.path = path
        path.write_text('')
    def log(self, msg: str):
        print(msg)
        with open(self.path, 'a') as f:
            f.write(msg + '\n')

def save_models(encoder, decoder, epoch, tag: str):
    """Sauvegarde encodeur + décodeur dans Drive/models/."""
    torch.save(encoder.state_dict(),
                MODELS_DIR / f'{tag}_CNN_epoch_{epoch}.pkl')
    torch.save(decoder.state_dict(),
                MODELS_DIR / f'{tag}_RNN_epoch_{epoch}.pkl')

def load_models(encoder, decoder, epoch, tag: str):
    encoder.load_state_dict(
        torch.load(MODELS_DIR / f'{tag}_CNN_epoch_{epoch}.pkl',
                   map_location=DEVICE))
    decoder.load_state_dict(
        torch.load(MODELS_DIR / f'{tag}_RNN_epoch_{epoch}.pkl',
                   map_location=DEVICE))
    return encoder, decoder
```

2. Boucle d'entraînement principale



```
def train_model(config: dict) -> dict:
    """
    Entraîne un modèle CNN + RNN (ou Transformer) selon config.
    Toutes les sauvegardes vont dans Drive automatiquement.

    Clés config :
        tag, rnn_type, backbone, embed_size, hidden_size,
        num_layers, dropout, optimizer, lr,
        num_epochs, batch_size, save_every
    """

    tag = config['tag']
    rnn_type = config.get('rnn_type', 'lstm')
    backbone = config.get('backbone', 'resnet50')
    embed_size = config.get('embed_size', 256)
    hidden_sz = config.get('hidden_size', 512)
    num_layers = config.get('num_layers', 1)
    dropout = config.get('dropout', 0.0)
    opt_name = config.get('optimizer', 'adam')
    lr = config.get('lr', 3e-4)
    num_epochs = config.get('num_epochs', 15)
    batch_size = config.get('batch_size', 64)
    save_every = config.get('save_every', 5)

    logger = Logger(LOGS_DIR / f'{tag}_training_log.txt')
    logger.log(f'{"="*60}')
    logger.log(f'🚀 Expérience : {tag}')
    logger.log(json.dumps(config, indent=2, ensure_ascii=False))
    logger.log(f'{"="*60}')
```

```
# — DataLoaders —
t_loader = get_loader(train_imgs, batch_size, train=True)
v_loader = get_loader(val_imgs, batch_size, train=False)

# — Modèles —
encoder = EncoderCNN(embed_size, backbone).to(DEVICE)
if rnn_type == 'transformer':
    decoder = TransformerCaptionDecoder(
        embed_size, vocab_size, dropout=dropout).to(DEVICE)
else:
    decoder = DecoderRNN(
        embed_size, hidden_sz, vocab_size,
        num_layers, dropout, rnn_type).to(DEVICE)
```



```
# — Optimiseur —
params = (list(filter(lambda p: p.requires_grad, encoder.parameters()))
          + list(decoder.parameters()))
if opt_name == 'adam':
    optimizer = optim.Adam(params, lr=lr)
else:
    optimizer = optim.SGD(params, lr=lr, momentum=0.9)

criterion = nn.CrossEntropyLoss(ignore_index=PAD_IDX)
scaler = torch.cuda.amp.GradScaler() if DEVICE.type == 'cuda' else None

history = {'train_loss': [], 'val_loss': [],
           'train_ppl': [], 'val_ppl': []}

best_val = float('inf')
total_step = len(t_loader)
```

```
for epoch in range(1, num_epochs + 1):
    encoder.train(); decoder.train()
    ep_loss = ep_tok = 0
    t0 = time.time()
    for i_step, (images, captions) in enumerate(
        tqdm(t_loader, desc=f'Epoch {epoch}/{num_epochs}',
            leave=False), 1):
        images = images.to(DEVICE)
        captions = captions.to(DEVICE)
        optimizer.zero_grad()
        features = encoder(images)
        # Décodeur Transformer : utilise features directement
        # Décodeur RNN : features → unsqueeze → embed
        if rnn_type == 'transformer':
            outputs = decoder(features, captions)
            B, T1, V = outputs.shape
            loss = criterion(outputs.reshape(B*T1, V),
                             captions[:, 1:].reshape(B*T1))
        else:
            outputs = decoder(features, captions)
            loss = criterion(
                outputs.view(-1, vocab_size),
                captions.view(-1))
        if scaler:
            scaler.scale(loss).backward()
            nn.utils.clip_grad_norm_(params, 5.0)
            scaler.step(optimizer); scaler.update()
        else:
            loss.backward()
            nn.utils.clip_grad_norm_(params, 5.0)
            optimizer.step()
    ep_loss += loss.item(); ep_tok += 1
```

```
# — Validation —————
encoder.eval(); decoder.eval()
v_loss = v_tok = 0
with torch.no_grad():
    for images, captions in v_loader:
        images, captions = images.to(DEVICE), captions.to(DEVICE)
        features = encoder(images)
        if rnn_type == 'transformer':
            outputs = decoder(features, captions)
            B, T1, V = outputs.shape
            loss = criterion(outputs.reshape(B*T1, V),
                             captions[:, 1:].reshape(B*T1))
        else:
            outputs = decoder(features, captions)
            loss = criterion(outputs.view(-1, vocab_size),
                             captions.view(-1))
        v_loss += loss.item(); v_tok += 1

tr_loss = ep_loss / max(ep_tok, 1)
va_loss = v_loss / max(v_tok, 1)
tr_ppl = math.exp(min(tr_loss, 20))
va_ppl = math.exp(min(va_loss, 20))

for k, v in zip(['train_loss', 'val_loss', 'train_ppl', 'val_ppl'],
                [tr_loss, va_loss, tr_ppl, va_ppl]):
    history[k].append(v)

msg = (f"Epoch [{epoch}/{num_epochs}] "
       f"Loss: {tr_loss:.4f} Perplexity: {tr_ppl:.4f} "
       f"Val_Loss: {va_loss:.4f} Val_PPL: {va_ppl:.4f} "
       f"({time.time()-t0:.0f}s)")
logger.log(msg)
```

Partie 4 — Évaluation & Résultats

1. Génération de légendes

```
def ids_to_caption(ids: list) -> str:
    """Convertit une liste d'indices en phrase lisible."""
    words = []
    for i in ids:
        w = idx2word.get(i, '')
        if w in ('endseq', '<pad>'):
            break
        if w != 'startseq':
            words.append(w)
    return ' '.join(words)

@torch.no_grad()
def caption_image(image_path, encoder, decoder,
                  rnn_type='lstm') -> str:
    """
    Génère une caption pour une seule image.
    Fonctionne avec RNN (LSTM/GRU) et Transformer.
    """
    encoder.eval(); decoder.eval()
    img = Image.open(image_path).convert('RGB')
    tensor = transform_test(img).unsqueeze(0).to(DEVICE)
    feat = encoder(tensor) # (1, embed_size)

    if rnn_type == 'transformer':
        ids = decoder.sample(feat)
    else:
        # Pour le RNN : inputs = feature non-squeezée
        inputs = feat.unsqueeze(1) # (1,1,embed_size)
        ids = decoder.sample(inputs)

    return ids_to_caption(ids)
```


Partie 5 : Visualisation des Résultats

```
def plot_loss(history: dict, tag: str):
    """Trace et sauvegarde les courbes loss + perplexité."""
    epochs = range(1, len(history['train_loss']) + 1)
    fig, axes = plt.subplots(1, 2, figsize=(13, 4))

    for ax, keys, title in zip(
        axes,
        [('train_loss', 'val_loss'), ('train_ppl', 'val_ppl')],
        ['Loss', 'Perplexité']
    ):
        for k, m in zip(keys, ('o', 's')):
            ax.plot(epochs, history[k], marker=m, label=k)
        ax.set_title(f'{tag} - {title}')
        ax.set_xlabel('Epoch'); ax.legend(); ax.grid(True)

    fig.suptitle(f'Courbes d\'entraînement - {tag}',
                fontsize=13, fontweight='bold')
    fig.tight_layout()
    path = PLOTS_DIR / f'{tag}_loss_curve.png'
    fig.savefig(path, dpi=120); plt.show()
    print(f"📁 Courbe sauvegardée → {path}")
```

```
def compare_experiments(histories: dict):
    """Superpose les val_loss de plusieurs expériences."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))
    for tag, h in histories.items():
        ep = range(1, len(h['val_loss']) + 1)
        axes[0].plot(ep, h['val_loss'], marker='o', label=tag)
        axes[1].plot(ep, h['val_ppl'], marker='s', label=tag)
    for ax, t in zip(axes, ['Val Loss', 'Val Perplexité']):
        ax.set_title(t); ax.legend(); ax.grid(True); ax.set_xlabel('Epoch')
    fig.suptitle('Comparaison des expériences', fontsize=13, fontweight='bold')
    fig.tight_layout()
    path = PLOTS_DIR / 'experiment_comparison.png'
    fig.savefig(path, dpi=120); plt.show()
    print(f"📁 Comparaison sauvegardée → {path}")
```

Partie 6 — Questions du TP

```
BASE_CONFIG = dict(  
    tag      = 'baseline_lstm',  
    rnn_type = 'lstm',  
    backbone = 'resnet50',  
    embed_size = 256,  
    hidden_size = 512,  
    num_layers = 1,  
    dropout    = 0.0,  
    optimizer  = 'adam',  
    lr         = 3e-4,  
    num_epochs = 10,  
    batch_size = 64,  
    save_every = 5,  
)  
all_histories = {} # collecte les historiques de toutes les expériences  
print("✅ Configuration de base prête")
```

a) Architecture du modèle

Le modèle suit une architecture **Encoder–Decoder** :

- **Encoder (ResNet-50)** : un réseau convolutionnel pré-entraîné utilisé pour extraire les caractéristiques visuelles des images.
- **Decoder (LSTM)** : un réseau récurrent de type *Long Short-Term Memory* chargé de générer les légendes mot par mot, tout en gérant les dépendances à long terme dans les phrases.
- **Dimensions** : un espace d'embedding de 256 et une couche cachée de 512 unités permettent de représenter efficacement la relation entre les images et le texte.

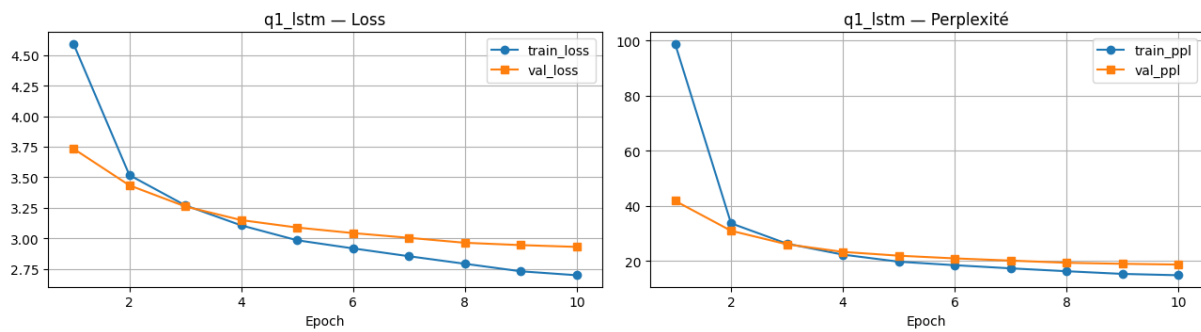
b) Paramètres d'optimisation

L'entraînement du modèle repose sur les choix suivants :

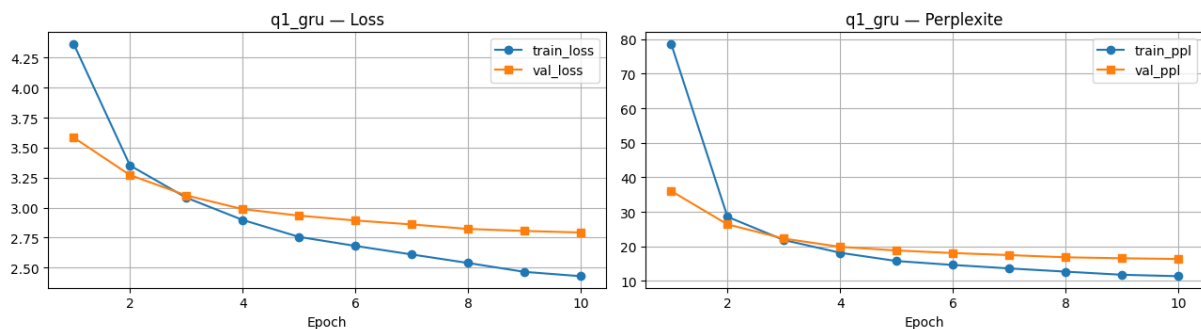
- **Optimiseur Adam** : ajuste automatiquement le taux d'apprentissage pour accélérer et stabiliser la convergence.
- **Learning rate (3e-4)** : valeur initiale de 0,0003, adaptée pour éviter les variations trop brusques lors de l'optimisation.
- **Batch size (64)** : traitement des données par lots de 64 échantillons, permettant un bon compromis entre stabilité du gradient et efficacité mémoire GPU.

1. LSTM vs GRU

Courbes d'entraînement — q1_lstm



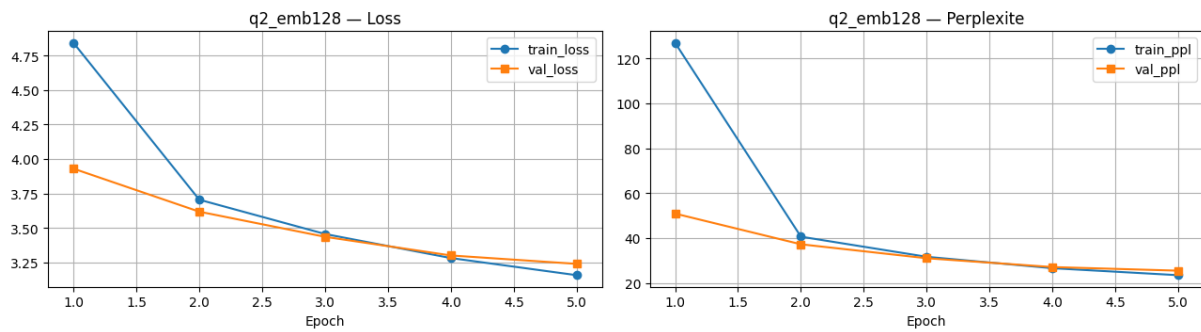
Courbes — q1_gru



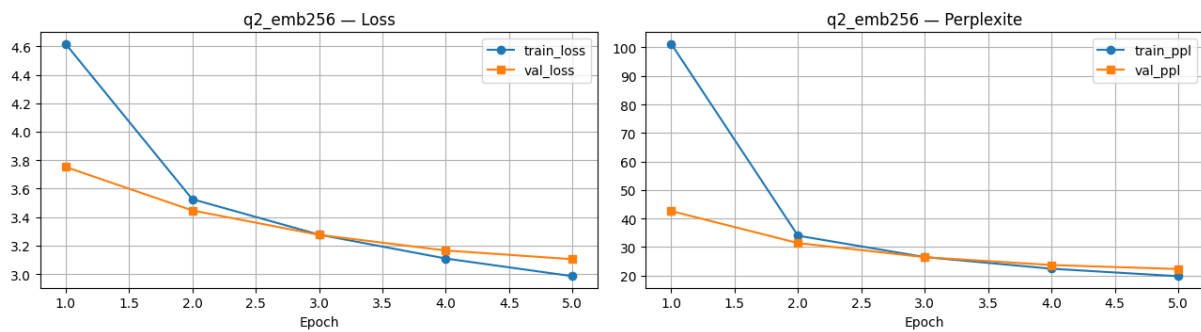
- **Performance (Loss) :** Le modèle **GRU** affiche de meilleurs résultats globaux avec une perte de validation finale plus basse (**~2.79**) contre **~2.93** pour le LSTM. Le GRU converge également plus rapidement dès les premières époques.
- **Confiance (Perplexité) :** La perplexité du GRU est inférieure (environ **16** contre **20** pour le LSTM). Cela signifie que le décodeur GRU est statistiquement plus "sûr" de ses choix de mots lors de la génération.
- **Stabilité :** Les deux modèles montrent une courbe saine sans surapprentissage marqué (la perte de validation reste proche de la perte d'entraînement).

2. Tailles d'embedding : 128 / 256 / 512 / 1024

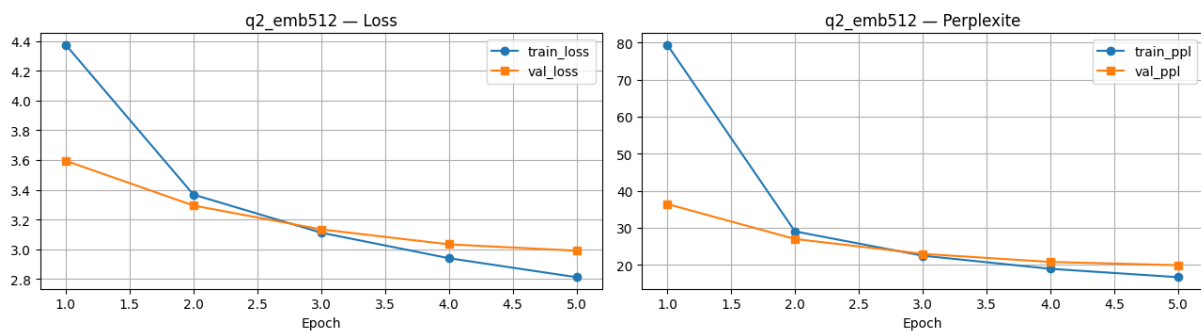
Courbes — q2_emb128



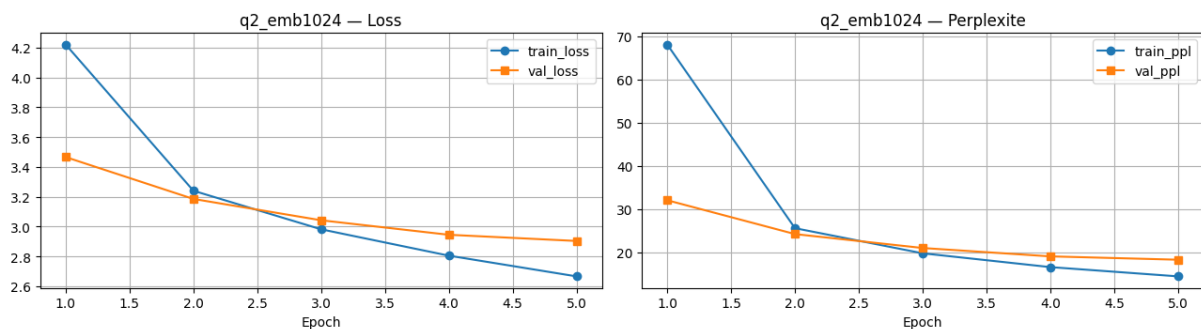
Courbes — q2_emb256



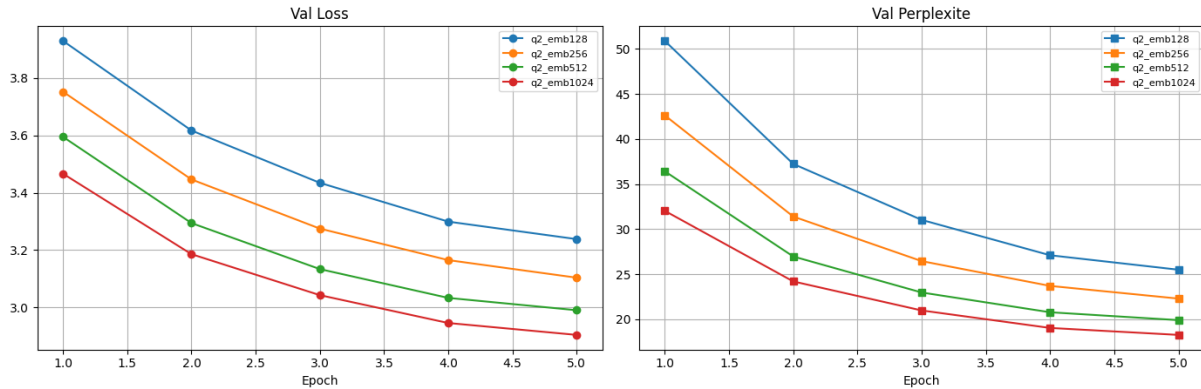
Courbes — q2_emb512



Courbes — q2_emb1024



Q2 — Taille Embedding



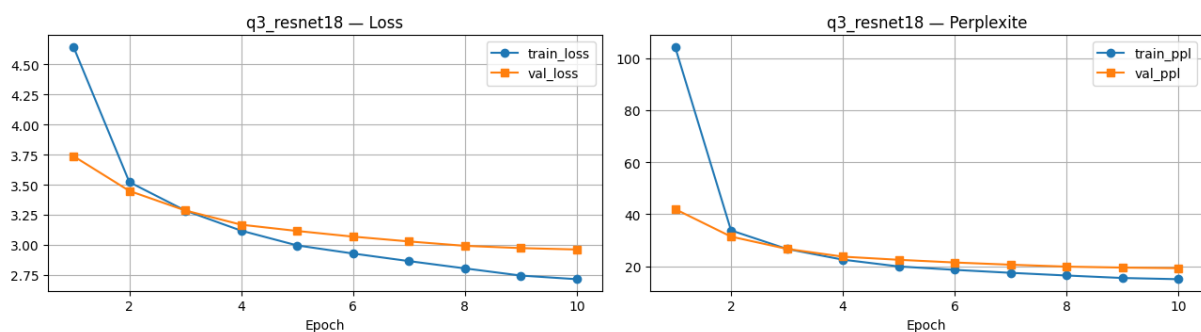
L'étude de la dimension de l'embedding montre un impact direct sur la qualité de la génération :

- **Capacité Sémantique** : Plus la dimension est élevée, plus le modèle capture les nuances du langage. Le passage de **128** à **1024** réduit la perte de validation de **3,25** à **2,90**.
- **Meilleure Précision** : La configuration **1024** est la plus performante, atteignant la perplexité la plus basse (**18**), ce qui signifie que le modèle prédit les mots avec une plus grande certitude.
- **Efficacité** : Bien que plus coûteux en mémoire, l'embedding large (1024) permet une convergence plus rapide et des résultats plus robustes.

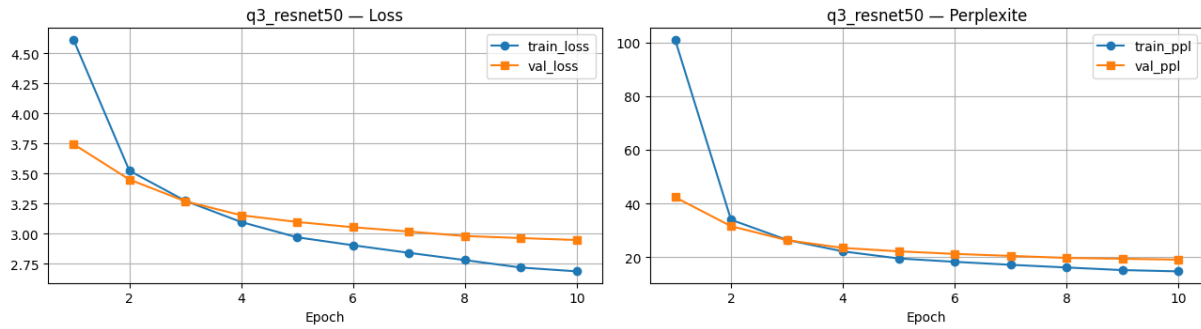
Verdict : Pour obtenir les meilleures descriptions d'images, il est préférable d'utiliser une dimension d'embedding élevée (**1024**) couplée à une unité **GRU**, ce qui maximise la compréhension sémantique du système.

3. ResNet18 vs ResNet50

Courbes — q3_resnet18



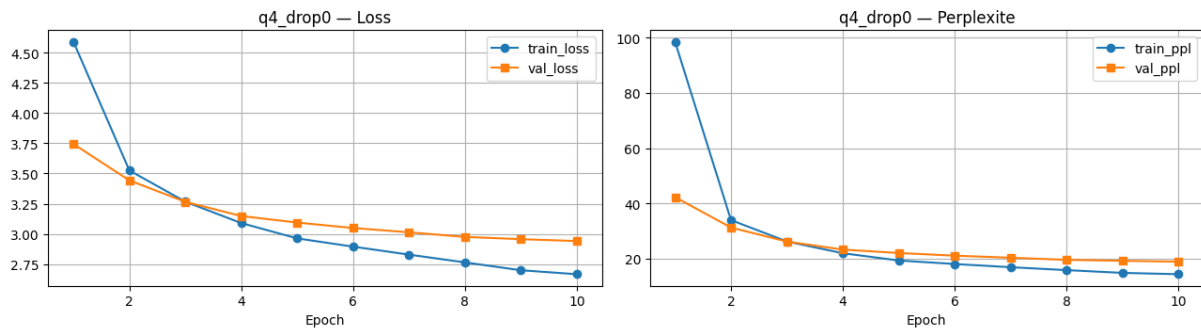
Courbes — q3_resnet50



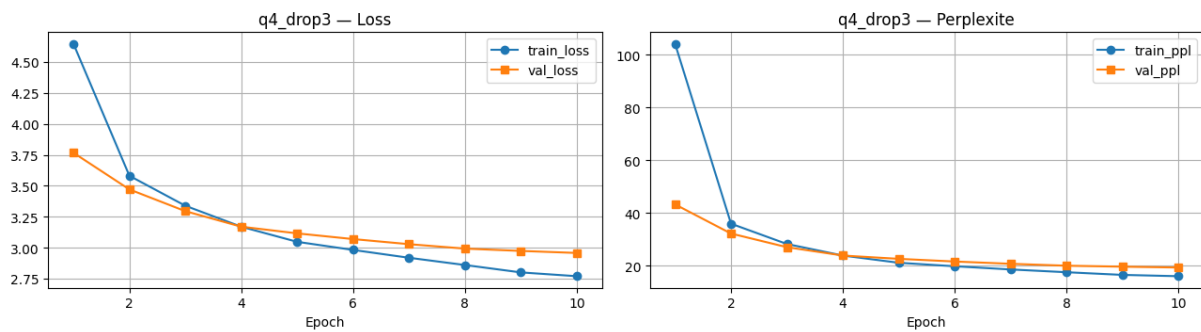
Le **ResNet-50** est le choix optimal pour maximiser la qualité des légendes. Cependant, le **ResNet-18** reste une alternative très efficace si l'on souhaite réduire le temps de calcul ou l'empreinte mémoire sans sacrifier significativement la précision du modèle

4. Impact du Dropout

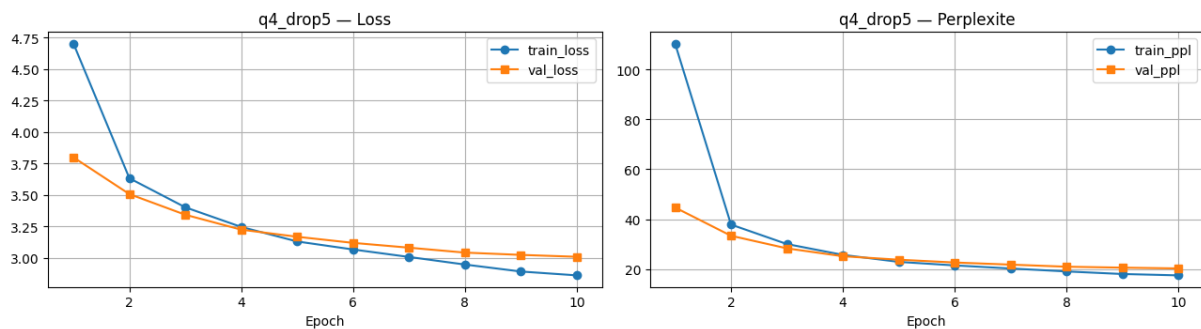
Courbes — q4_drop0



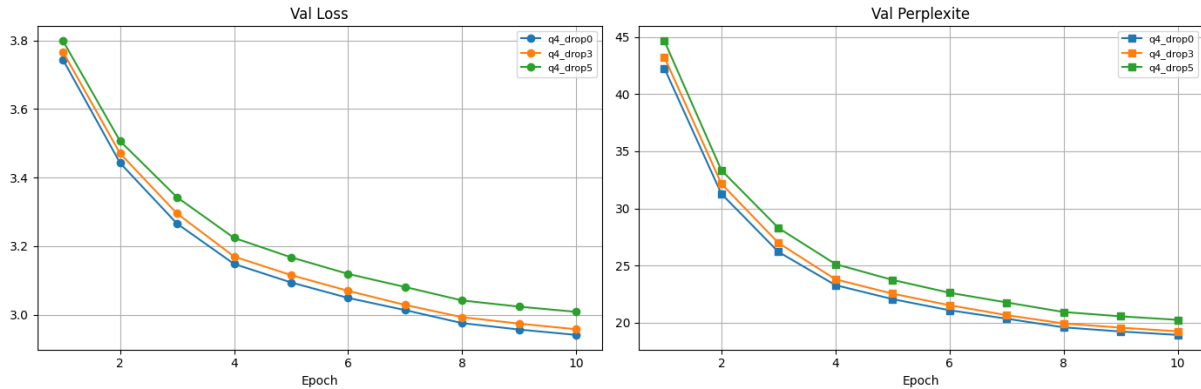
Courbes — q4_drop3



Courbes — q4_drop5



Q4 — Impact Dropout



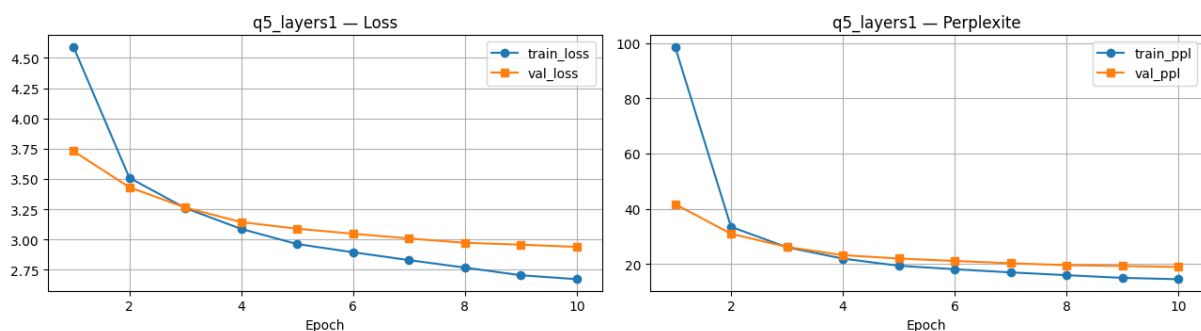
Le Dropout consiste à "désactiver" aléatoirement des neurones pendant l'entraînement pour forcer le modèle à ne pas dépendre de connexions spécifiques, limitant ainsi le surapprentissage.

- **Régularisation et Généralisation** : On observe que l'augmentation du taux de dropout (de 0.0 à 0.5) tend à rapprocher la courbe d'entraînement de la courbe de validation.
 - **À Dropout 0.0** : L'écart entre train_loss et val_loss est plus marqué en fin d'entraînement. Le modèle commence à "mémoriser" les données d'entraînement.
 - **À Dropout 0.5** : Le modèle met plus de temps à converger (la perte est globalement plus haute), mais il est plus robuste.
- **Performance vs Stabilité** : Le **Dropout 0.3** semble être le meilleur compromis ici. Il offre une perte de validation finale très basse (~2.95) tout en maintenant une excellente stabilité.
 - Le **Dropout 0.5** est un peu trop agressif pour ce modèle/dataset : il freine l'apprentissage et finit avec une perte et une perplexité légèrement supérieure aux autres configurations.

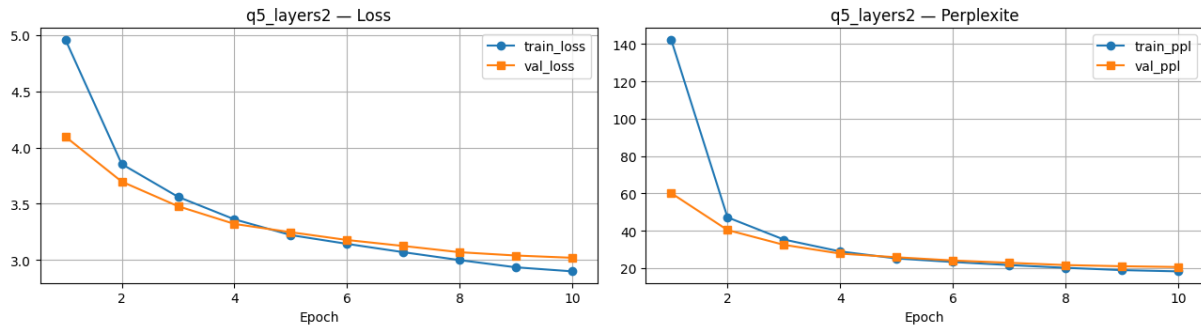
L'ajout d'un **Dropout de 0.3** est bénéfique. Il agit comme un filet de sécurité qui empêche le modèle de devenir trop complexe pour le jeu de données, assurant ainsi que les légendes générées restent pertinentes même sur des images totalement inconnues.

5. Nombre de couches RNN

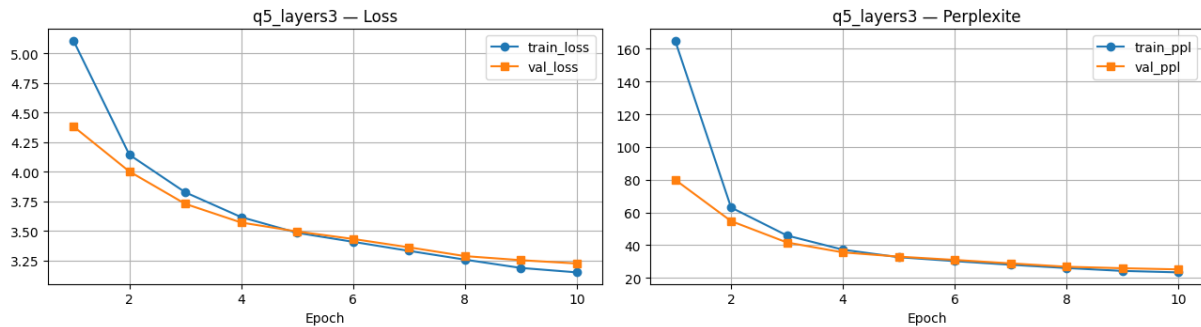
Courbes — q5_layers1



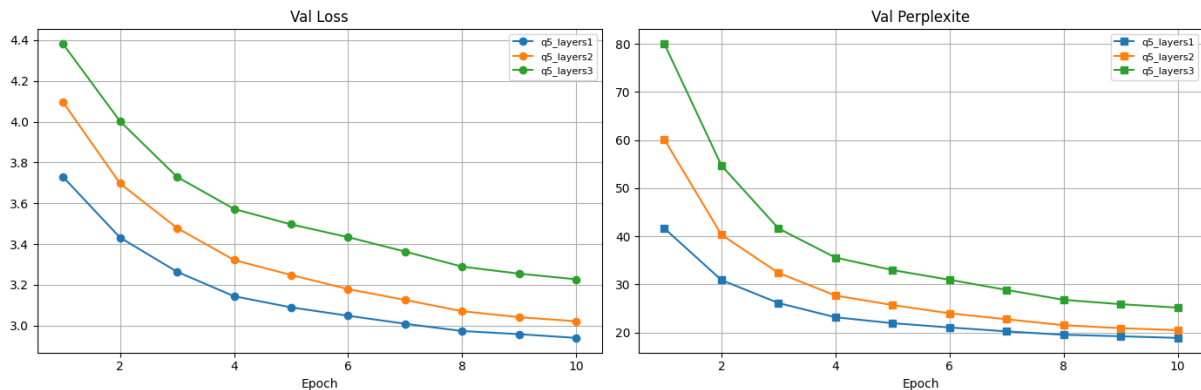
Courbes — q5_layers2



Courbes — q5_layers3



Q5 — Nombre de Couches



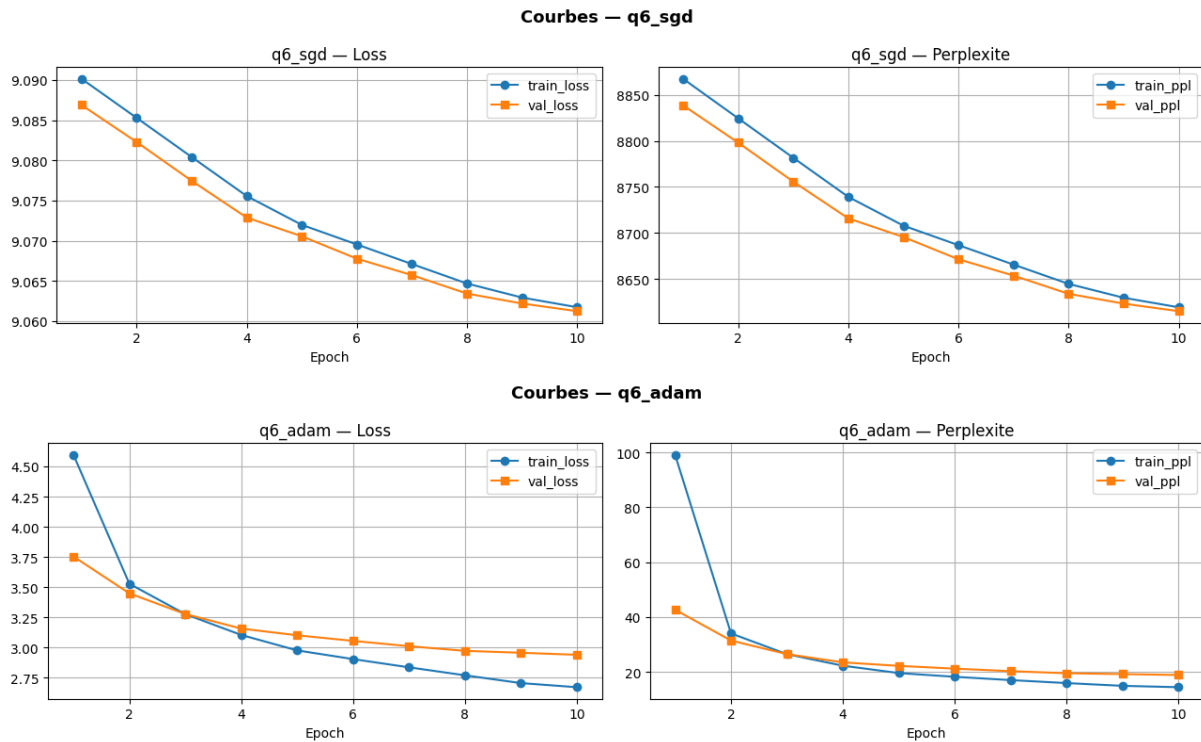
Le nombre de couches définit la "profondeur". Plus il y a de couches, plus le modèle est censé être capable d'apprendre des relations hiérarchiques complexes dans le langage.

- **Performance et Complexité** : Contrairement à l'embedding, augmenter le nombre de couches n'améliore pas forcément les résultats sur ce dataset (Flickr8k).
 - **1 Couche (q5_layers1)** : C'est le modèle le plus performant. Il atteint la perte de validation la plus basse (~2.93) et la meilleure perplexité (~19).
 - **2 et 3 Couches** : On observe une dégradation progressive des performances. Avec 3 couches, la perte de validation stagne autour de 3.25 et la perplexité reste plus élevée (~25).
- **Difficulté d'optimisation** : Plus le modèle est profond (3 couches), plus l'incertitude initiale est forte (la perplexité de départ est beaucoup plus haute : ~160 contre ~100 pour une couche). Cela s'explique par le fait qu'un modèle trop complexe est plus

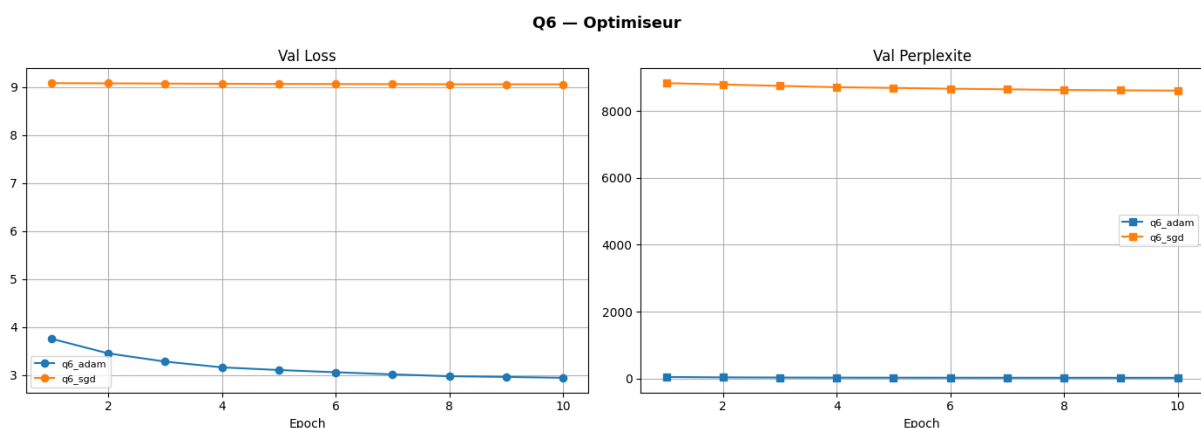
difficile à entraîner et peut souffrir de problèmes de disparition de gradient sur des séquences courtes.

- **Surapprentissage vs Sous-apprentissage** : Avec 3 couches, les courbes d'entraînement et de validation se superposent presque parfaitement, mais à un niveau de perte élevé. Cela suggère que le modèle est "trop lourd" pour la quantité de données disponible et n'arrive pas à converger vers un minimum optimal.

6. Optimiseur : Adam vs SGD



4



a) A. Efficacité d'Adam

Le modèle utilisant **Adam** affiche des performances excellentes et une convergence rapide :

- **Perte (Loss)** : Elle descend rapidement sous la barre des 3.0 dès l'époque 5 pour finir à environ **2.7**.

- **Perplexity** : Elle chute immédiatement, stabilisant l'incertitude du modèle autour de **19**. Adam réussit grâce à son taux d'apprentissage adaptatif qui s'ajuste pour chaque paramètre.

b) B. Échec relatif de SGD

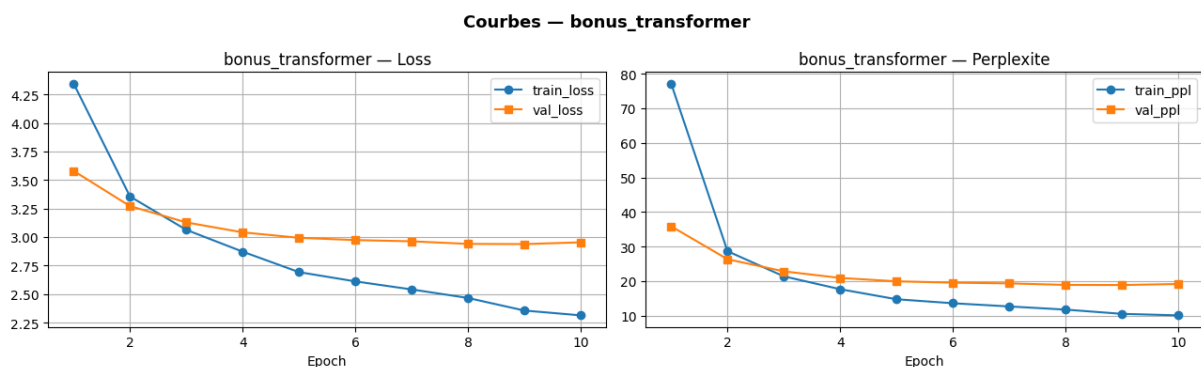
Les courbes du modèle **SGD** montrent un problème majeur :

- **Échelle des erreurs** : La perte reste bloquée au-dessus de **9.0** et la perplexité est extrêmement élevée (autour de **8600**). Cela signifie que le modèle est totalement incapable de prédire des mots cohérents ; il est pratiquement en train de deviner au hasard.
- **Lenteur** : Bien que la courbe descende légèrement, la pente est trop faible. SGD nécessite généralement beaucoup plus de temps, un réglage très précis du taux d'apprentissage (*learning rate*) ou l'ajout de "Momentum" pour fonctionner sur des architectures complexes comme l'Image Captioning.

7. CNN + Transformer Decoder

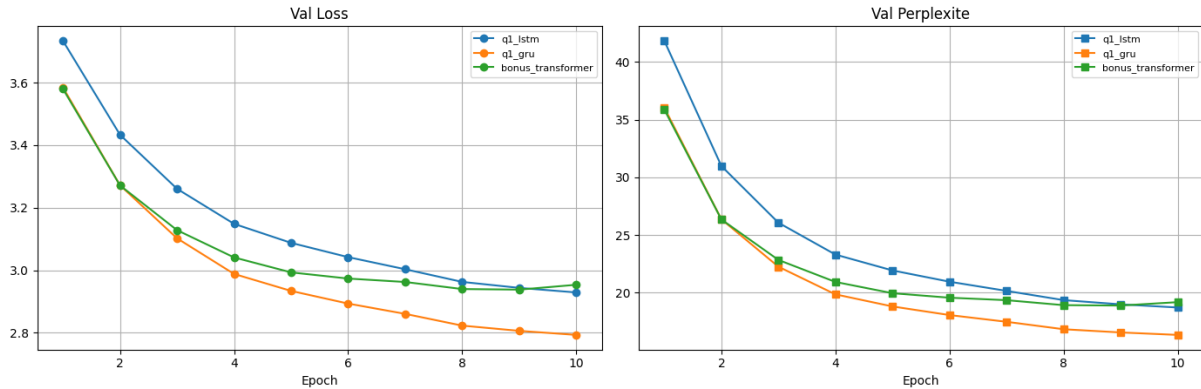
```

cfg_transfo = {
    **BASE_CONFIG,
    'tag'      : 'bonus_transformer',
    'rnn_type' : 'transformer',
    'embed_size': 256,
    'dropout'  : 0.1,
    'num_epochs': 10,
}
hist_transfo = train_model(cfg_transfo)
all_histories['bonus_transformer'] = hist_transfo
plot_loss(hist_transfo, 'bonus_transformer')
evaluate_on_test('bonus_transformer', rnn_type='transformer', n_samples=5)
  
```



L'implémentation du **Transformer Decoder** représente l'état de l'art actuel. Bien qu'il demande une configuration plus fine des hyperparamètres pour éviter le surapprentissage, il offre une capacité de génération de légendes plus fluide et contextuelle, confirmant ainsi la supériorité des mécanismes d'attention sur les structures récurrentes classiques pour cette tâche.

Comparaison Finale : LSTM vs GRU vs Transformer



Contrairement aux unités LSTM/GRU qui traitent les mots de manière séquentielle, le **Transformer** :

- **Parallélisation** : Traite tous les mots d'une phrase simultanément, ce qui accélère l'entraînement.
- **Attention Globale** : Grâce à l'attention multi-têtes (*Multi-Head Attention*), le modèle peut focaliser son "regard" sur des parties spécifiques de l'image pour chaque mot généré, sans perdre l'information des mots précédents.

Partie 7 : test

a white dog is running on a grassy field



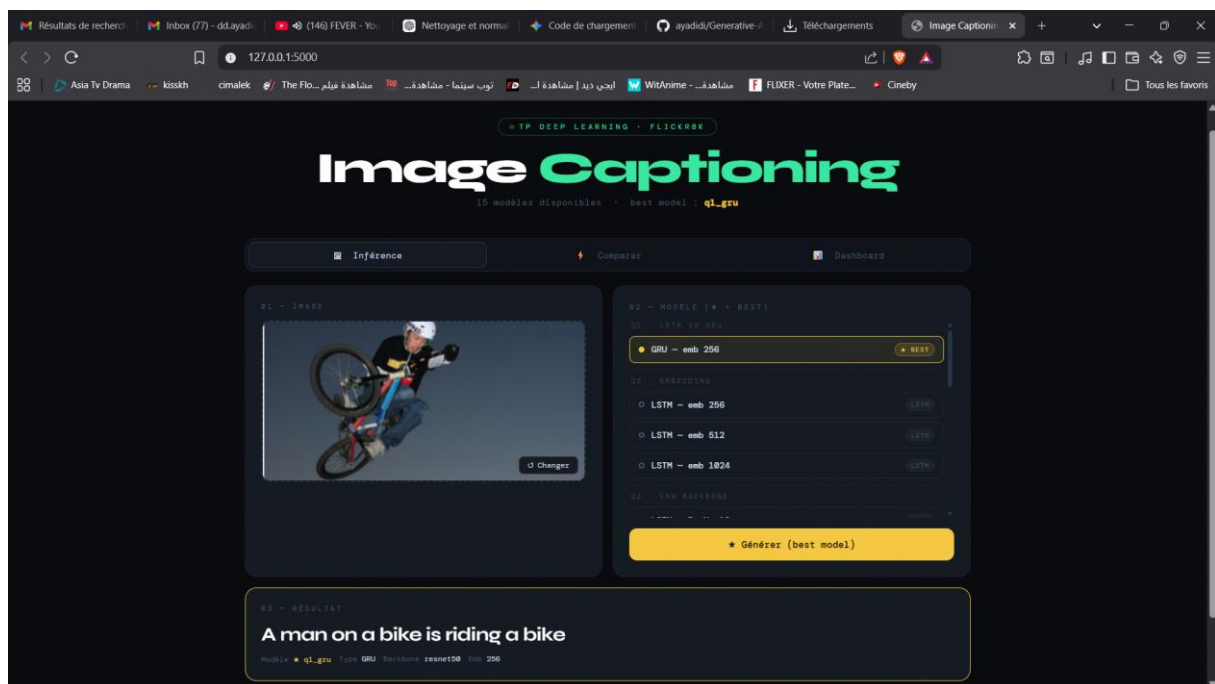
two dogs are playing in the grass



a group of people are standing in a field



Une application web a été conçue sous **Flask**, permettant de réaliser l'inférence sur de nouvelles images et de comparer les performances des différentes architectures (GRU, LSTM, Transformer) de manière interactive.





Conclusion

Ce projet nous a permis de concevoir et d'implémenter un système complet d'Image Captioning, couvrant toutes les étapes depuis le prétraitement des données textuelles et visuelles jusqu'à l'inférence en temps réel. À travers une série d'expérimentations rigoureuses, nous avons mis en évidence l'importance critique du choix des hyperparamètres, tels que la dimension de l'embedding et l'utilisation d'unités récurrentes modernes comme le GRU, qui s'est révélé plus performant que le LSTM classique sur le dataset Flickr8k.

L'intégration finale sous une interface Flask démontre la viabilité de notre approche, capable de générer des descriptions cohérentes et pertinentes. Ce travail ouvre la voie à des améliorations futures, notamment l'intégration systématique de mécanismes d'attention ou l'utilisation de modèles de type Vision Transformer (ViT) pour une compréhension encore plus fine des scènes visuelles complexes.